



Budapesti Műszaki és Gazdaságtudományi Egyetem
Villamosmérnöki és Informatikai Kar
Irányítástechnika és Informatika Tanszék

Általános célú programnyelv szakterület-specifikus nyelvek megvalósítására

TDK dolgozat

Készítette:

Bodor András

Konzulens:

Dr. Simon Balázs

2025

Tartalomjegyzék

Kivonat	i
Abstract	ii
1. Bevezetés	1
2. Szakterület-specifikus nyelvek	3
3. A C4 programozási nyelv	4
3.1. A C4 nyelv leírása	4
3.1.1. Az elvárt tulajdonságok	4
3.1.2. A C4 nyelv ismertetése	5
3.2. A c4c és c4i megvalósítása	6
3.2.1. Felhasznált eszközök	6
3.2.2. Átfogó architektúra	7
4. Használati példák	8
4.1. Állapotgép leíró nyelv	8
5. Jövőbeli fejlesztések és tervek	10
5.1. Paraméter-kötött függvények	10
5.2. Scriptelhető statikus analízis eszköz	11
Irodalomjegyzék	12
Függelék	13
F.1. A TeXstudio felülete	13
F.2. Válasz az „Élet, a világmindenség, meg minden” kérdésére	14

Kivonat

A szakterület-specifikus nyelvek (DSL) az informatika számos részén jól használható és kényelmes lehetőséget nyújtanak egy-egy speciális feladatkör megoldására. Manapság számos olyan eszköz elérhető, amely DSL-ek fejlesztésére alkalmas, de kevés olyan programnyelv létezik, amelybe a DSL-ek kényelmesen beágyazhatók.

A dolgozatban egy általam kifejlesztett általános célú programnyelvet mutatok be, amelyben – a nyelv kialakításának köszönhetően – könnyedén lehet DSL-eket definiálni és futtatni. A nyelv fordítója LLVM technológiára épít, így akár JIT segítségével, akár teljes natív fordítással lehetővé teszi a megvalósított DSL-ben írt programok futtatását.

A dolgozatban demó jelleggel bemutatok néhány gyakorlatban is használható DSL-t, valamint egy nagyobb léptékű jövőbeli bővítési tervet.

Abstract

Domain-specific languages (DSLs) are useful in numerous fields of computer science and provide a convenient way to solve a specific set of tasks. Today there are many tools available for developing DSLs, but few programming languages exist into which DSLs can be easily embedded.

In this paper, I present a general-purpose programming language that I developed, in which DSLs can be easily defined and executed thanks to the way the language is designed. The language's compiler is based on LLVM technology, so it allows programs written in the implemented DSL to be executed either with JIT or with full native compilation.

As a demonstration, I present a few DSLs that can be used in practice, as well as a larger-scale plan for future expansion.

1. fejezet

Bevezetés

Az informatika széles körben alkalmaz különféle nyelveket ahhoz, hogy valamilyen feladatra tudjon megoldást adni: gyakran ez egy program fejlesztése, ahol egy megfelelően választott programozási nyelvet (pl. C++, C#, Java, Rust, Haskell, Erlang, stb.) szokás használni – a megfelelő választás pedig függ a feladat jellegétől, a projekttel szemben támasztott követelményektől és magától a fejlesztő csapat előképzettségétől és képességeitől is.

Egy nagyszerűen a problémára illeszkedő programozási nyelv választásakor is előfordul azonban, hogy előkerülnek olyan részfeladatok, amelyekhez nincs szükség az előbb felsorolt programnyelvek teljes eszközkészletére, hogy megoldjuk ezeket. Ilyenkor az is lehetséges, hogy ez az eszköztár akár még meg is nehezítheti a konkrét eset megoldását és karbantarthatóságát, mert túl sok lehetőséget nyújtanak a fejlesztőknek, így egy olyan megoldás áll elő amelyik túlságosan komplex, pedig lett volna lehetőség egy letisztultabb megoldásra is – ez egyszerűen nem jutott a fejlesztők eszébe a túlságosan tág megoldási lehetőségek halmazából.

Ez az egész problémakör azért merül fel, mert ezek a nyelvek ún. általános célú programozási nyelvek (General-purpose Programming Language, GPL), amelyek célja és tervezésükkor kitűzött alapvető elvárás, hogy minden feladatra és problémára amivel szembe találkozhat egy fejlesztő, arra legyen valamilyen lehetősége azt a nyelven belül megoldani. Így minden esetben minden probléma megoldásához szükséges minden lehetőséget biztosítani, akkor is, ha például egy állapotgép leírásához nincs feltétlenül szükségünk arra, hogy tudjunk osztállysablonokat definiálni.

Ez egy gyakran előforduló helyzet, hiszen sok esetben a nagy célprobléma részekre bontása során előállnak olyan részproblémák, amelyek az előző paragrafusban tárgyalt túlságosan gazdag leírási lehetőségek szerint a nem annyira komplex és kevésbé általános nyelvi keretek könnyítenék a fejlesztést. Ezzel összhangban, sok olyan speciális nyelv alakult ki, amelyik az általános programnyelvek teljes eszköztárát limitált formában, vagy egyáltalán nem biztosítja. Ezekért cserébe a megcélzott szakterület igényeit elégíti ki a lehető legjobban. Egy kifejezetten gyakran előkerülő család a leíró nyelvek (XML dialektusok, HTML, Markdown), de igen nagy számban fordulnak elő különböző egyéb példák: ábrák készítésére készült nyelvek (Pikchr, PlantUML), fordítást kezelő rendszerek (Make, Ninja), de még az adatmanipulációs SQL és QUEL nyelvek és tipográfiai nyelvek, mint a $\text{\TeX}$ és roff is ebbe a kategóriába tartoznak.

Az előző bekezdésben tárgyalt nyelveket, amelyek egy-egy részprobléma megoldására adnak specializált – és pusztán arra limitált – eszközkészletet hívjuk szakterület-specifikus

nyelveknek (Domain Specific Languages, DSL). Ezek részletesebb tárgyalásáról szól a 2. fejezet.

Az általam fejlesztett általános célú programozási nyelvet és annak általam adott megvalósítását mutatja be a 3. fejezet. Itt ismertetem a tervezési elveket, és hogy milyen elvárásoknak megfeleléssel készült a C4 nyelv. A használt eszközöket felvezetem, és bemutatom a megvalósítás általános struktúráját, kiemelve az érdekesebb, kevésbé elterjedt nyelvi elemekhez tartozó konkrét megvalósítási lépéseket.

A 4. fejezetben pedig a megvalósított nyelvhez mutatok a valóságban is használható felhasználási lehetőséget: megvalósított szakterület-specifikus nyelveket, amelyekben készített kódrészleteket egy az egyben futtatni is lehet a C4 nyelv fordítóját felhasználva.

Ezek után az 5. fejezetben ismertetem a jövőbeli továbbfejlesztési lehetőségeket, és egy olyan nagyobb lélegzetvételű C4 nyelvet felhasználó projektet, amelyik C és C++ kódoknak kódolható statikus analízisét teszi lehetővé.

2. fejezet

Szakterület-specifikus nyelvek

3. fejezet

A C4 programozási nyelv

3.1. A C4 nyelv leírása

3.1.1. Az elvárt tulajdonságok

Az alábbi tulajdonságok azok, amelyet a nyelvvel kapcsolatban felállítottam a tervezés előtt. Ezek egy általános vázat adnak a végső nyelv viselkedésével kapcsolatban, és garantálják az elvárt módon való használhatóságot a nyelvtől. A tulajdonságok számozása egy gyenge fontossági sorrendet valósít meg köztük: bár mindegyik fontos, de az első nem teljesítése sokkal nagyobb probléma, mint a 4.-el szemben tapasztalt esetleges problémák.

Tulajdonság 1. Erős beágyazott DSL támogatás

A beágyazott DSL-ek létrehozásának lehetősége az egyik alapkoncepció a nyelv tervezése során. Egy erősen behatárolt, nem rugalmas vagy csak nem eléggé egységesített szintaxisú nyelv esetén limitálódik azon DSL-ek szintaktikája is, amit a nyelvbe lehet beágyazni. Ennek megfelelően az egyik legfontosabb tulajdonsága a nyelvnek, hogy a lehető legrugalmasabb szintaktikát megválasztva, a DSL-eket legkevésbé korlátozva azok létrehozását megfelelő módon támogassa.

Tulajdonság 2. Fordítás nélkül és fordítással is futtatható.

A gazdanyelvbe sokféle DSL beágyazásának a lehetősége adott. Ezek a DSL-ek viszont magukban is rendelkezhetnek különböző tulajdonságokkal, amelyek számukra előnyösebbé teszik a direkt kiértékelést (interpretálást) vagy fordítást a velük végzett munka során. Például a fordítás előnyös lehet, ha olyan végterméket állítunk elő, amelyet a végfelhasználó számára csomagolt binárisként szállítunk le így nem kell a teljes interpreter eszköztárát is telepíteni, de interpretálással rövid, egyszer használatos programokat könnyebb készíteni és használni. Egy-egy DSL-nek mind két esetben is használhatónak kell maradnia, így a nyelvi szintű támogatás fontos, hogy megjelenjen mind kettő lehetőséghez.

Tulajdonság 3. Dinamikusan típusos

Számos érv szól mind a statikus és dinamikus típusos nyelvek mellett, emellett egyes programozóknak vannak személyes preferenciái, hogy melyikkel szeretnek jobban dolgozni; nem lehet egy egyértelműen kiválasztani „a jó” opciót a kettő közül. Azért a dinamikusan típusos megoldásra esett a választásom, mert így a DSL-ek használata közben nem kell a potenciálisan komplex típusok megadásával (fejlesztői oldalon), vagy kikövetkeztetésével (fordító oldalon, ahol egyáltalán egyértelműen lehet) sok időt és energiát eltölteni. Elegendő, csak a szakterület által is érdekesnek ítélt elemekkel foglalkozni.

Tulajdonság 4. Érthető egy átlagos C, C++, Java, stb. fejlesztő számára.

A nyelv szintaktikájának lehetőleg a megszokott, közismert nyelvek szintaktikájához lenne előnyös illeszkednie, hogy egy olyan programozó, amelyik már egy elterjedt másik nyelvet ismer, ne kelljen a szintaktikának a tanulmányozásával sok időt töltenie; legyen elég, ha a szemantikával megismerkedik, és utána szinte azonnal képes legyen használni a nyelvet, vagy egy benne megalkotott DSL-t.

3.1.2. A C4 nyelv ismertetése

Az előző tulajdonságok alapján megalkotott nyelv leírása következik. Magas szinten, egy olyan nyelvet definiálok, amelyik funkcionális, immutábilis, dinamikus típusos, és szintaktikáját tekintve nagyon megengedő és rugalmas.

Ezeknek a tulajdonságoknak nagy része egyenesen megfeleltethető vagy direkt következménye az előző részben definiált elvárásoknak. A funkcionális, immutábilis nyelv választása azonban nem egyértelmű: a klasszikus könnyen futtatható, dinamikus nyelvek nagy többségben egyszerű procedurális paradigma szerint működnek, ld. Tcl és a különböző (UNIX) shell nyelvek, esetleg utólagosan különféle módon objektum-orientált támogatást kaptak; ilyen pl. a Perl.

Az immutábilis entitásokkal végzett programozás biztosít egy extra réteg fordítási időbeli védelmet nyújt az esetleges elírások, vagy imperatív kódban ejthető logikai hibákkal szemben. Ez a megközelítés a dinamikus típusozás által hozott extra hibalehetőségeket hivatott csökkenteni, amellet, hogy több előnye is származik a kódnak a tisztaságából (purity): egyszerűbb értelmezés, és meglepő mellékhatások nélkül könnyebben olvashatóvá válhatnak kódrészletek.

A funkcionális paradigma pedig jobban illeszkedik egy ilyen teljesen immutábilis rendszerre, mint a procedurális vagy objektum-orientált; a funkcionális nyelvek nagy része teljesen immutábilis, és semmiféle értékmódosítást nem engedélyez. Emiatt jobbnak vélem, ha egy kevésbé ismert paradigmát választok, amelyik cserébe sok olyan mintát és idiómát tud más nyelvekből, hozni, amelyek alaptól hasonló koncepciók mellett működnek.

A logikai paradigmát még meg lehet említeni, mint egy paradigma, amelyik az immutábilis elemekkel jól komponálható. Azonban ez sok programozó számára idegen, és akár még kényelmetlen is tud lenni a Prolog jellegű kódolási stílus, ha az adott felhasználó nem rendelkezik megfelelő gyakorlattal. Ez direkt módon szegi a 4. tulajdonságot, amelyiket teljesíteni kell a nyelvnek.

A nyelv rugalmasságát viszont még az eddig elmondottak nem garantálják. Meg kell tehát még valósítani azt a nyelvi szintaktikát, amelyikben a lehető legkevesebb konkrét elem található, és minél több minden ezekből felépíthető. Ez a rugalmasság LISP nyelvcsaládban[5] kiemelkedően jól látszik: a zárójeleket leszámítva, makrók felhasználásával a Scheme implementációkban akár teljesen új nyelvi elemeket lehet[3] létrehozni. A zárójelek azonban egy idegen, elriasztó kinézetet kölcsönöznek a LISP alapú nyelveknek, így nem feltétlen ideálisak a 4. tulajdonság miatt. Ennek ellenére viszont a LISP alapkoncepciója hasznos tud lenni: az ún. szimbolikus kifejezés (`glsexpr`; innen `glsexpr`), amelyik az elkülönülő zárójeles kinézetét adja. Ebben láthatóan mindent függvények vagy makrók definiálásával és azok hívásával lehetséges megadni, és kiemelkedően egyszerű feldolgozni, mert minden teljesen uniform módon néz ki.

De vegyük el a zárójeleket: ha előre ismert, hogy mennyi paramétert vár egy adott függvény, akkor a zárójelek halmozása nélkül is lehetséges, hogy egyszerűen és megfelelően adjunk meg egy zárójelezést minden kifejezéshez. Ez egy C jellegű sorrendet követel meg a függvények definiálásánál: ez azonban nem okoz véleményem szerint nagy gondot, sok másik nyelv rendelkezik ezzel a megköttéssel: C, C++ (részei), Perl, stb.

Ezzel viszont egy olyan opcionálisan zárójelezhető LISP-et kapunk, amelyik kísértetiesen hasonlít egy megszokott C-szerű programozási nyelvre. Erről szól a 3.1. ábra.

```
(if (> a 2)
  (display "nagyobb")
  (display "kisebb"))
```

(a) Scheme kód if függvénynyel

```
if a > 2 {
    println "nagyobb"
} else {
    println "kisebb"
}
```

(b) C4 kód if függvénynyel

```
if (a > 2) {
    puts("nagyobb");
} else {
    puts("kisebb");
}
```

(c) C kód if konstrukcióval

3.1. ábra. Különböző nyelvek if szintaktikája

3.2. A c4c és c4i megvalósítása

Ebben az alfejezetben az előzőekben megadott C4 nyelvhez készített megvalósításomat, a c4c fordítóprogramot és c4i JIT alapú interpretert mutatom be. Elsőként a felhasznált eszközökről esik röviden szó, majd a az általános architektúra utána az érdekesebb megoldásokra térek ki.

3.2.1. Felhasznált eszközök

Az alábbi felsorolásban a fontosabb, nyelv megvalósításához használt eszközöket és könyvtárakat mutatom be röviden.

C++ A C4 nyelv megvalósítását a C++ nyelv C++23-as verziójában valósítottam meg. Ez egy közismert, natív kódra forduló, rendszerprogramozási nyelv.

PCRE2 [6] A lexerben használt reguláris kifejezéseket a PCRE2 könyvtár segítségével futtatom a bementi szövegen. Ennek érdekessége, hogy támogat a reguláris kifejezések értelmezéséhez egy JIT alapú megvalósítást, amelyikkel gyorsabban tudja futtatni a lefordított kifejezéseket, mint egyszerű naiv kiértékeléssel.

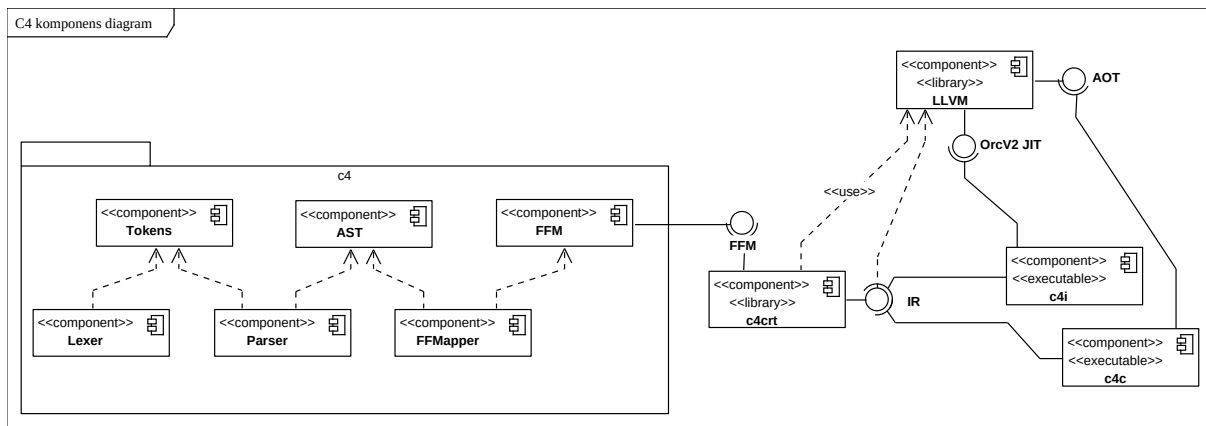
LLVM [4] Egy széles körben natív nyelvek megvalósítására használt eszközkészlet. A projekt saját fejlesztésű C, C++ és Fortran fordítója mellett a Rust és Swift programozási nyelvek is ezt használják a natív kód generálás megvalósítására. A C4 rendszer számára biztosítja a bináris kimenet generálását, optimalizálást és a JIT támogatást.

Boehm-Demers-Weiser GC [2] Egy hatékony[1] C könyvtárként megvalósított konzervatív szemétgyűjtő (garbage collector) algoritmus. Ez biztosítja, hogy a C4 feldolgozása során generált futtatható kód (akár külön binárisként, vagy interpretált módban) érvényes memóriára mutat. A különböző nyelvi elemek (closure) nehezé teszik az egyértelmű

memória foglalást manuálisan megvalósítani, így a szemétgyűjtő algoritmus jelentősen megkönnyíti a megvalósítást.

3.2.2. Átfogó architektúra

A C4 fordító és értelmező programok a kód nagy részében azonosak: a különbség, hogy a folyamat végén található program az LLVM által biztosított bináris kimenetet generál vagy az LLVM JIT lehetőségeit kihasználva direkt módon kezdi el kiértékelni a C4 kódnak megfelelő memóriabeli reprezentációt.



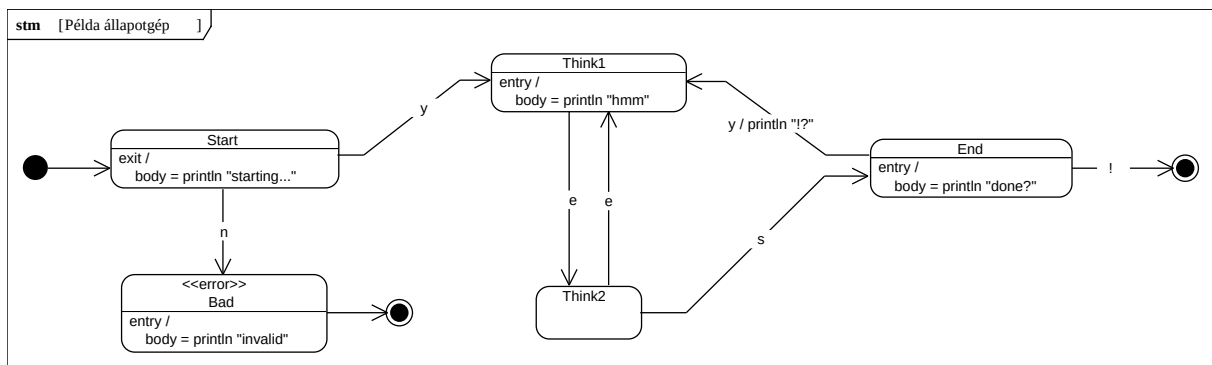
3.2. ábra. A C4 projekt átfogó ábrája

4. fejezet

Használati példák

4.1. Állapotgép leíró nyelv

A sokak által ismert és egyszerű véges állapotgépek (Finite State Machine (FSM)) leírására adott szakterület-specifikus nyelv jó példa tud lenni egy DSL készítésére képes eszköz számára. Ez többek között azért van, mert ezeket a matematikai jellegű struktúrákat egy deklaratív, letisztult, egyszerű nyelvben is le lehet írni szépen és elegánsan. Ez kifejezetten egy előnye egy DSL-nek, míg egy általános célú nyelvben ezt gyakran jóval bonyolultabban lehet megvalósítani. Ebben a szekcióban, az előzőeknek megfelelően, én is megadok egy FSM megvalósítást, amelyikkel egyszerű véges automatákat lehetséges megvalósítani. A konkrét megvalósító C4 kód függelékben, egy példaként megvalósított állapotgép látható a 4.1. ábrán. A DSL-ben írt kód a pedig alább következik a 4.1. listázásban.



4.1. ábra. A megvalósított állapotgép

```
1 let myfsm fsm
2   start state "start"
3     leave { println "starting..." }
4
5   final state "done"
6
7   error state "bad"
8     enter { println "invalid" }
9
10  state "think1"
11    enter { println "hmm" }
12
13  state "think2"
14
15  state "end"
16    enter { println "done?" }
```

```

17
18   on "n" from "start" to "bad"
19   on "y" from "start" to "think1"
20
21   on "e" from "think1" to "think2"
22
23   on "e" from "think2" to "think1"
24   on "s" from "think2" to "end"
25
26   on "y" from "end" to "think1"
27     do { println "!" }
28   on "!" from "end" to "done"
29 done

```

4.1. lista. C4 állapotgép DSL használati példa

Ami a C4 előnye ebben az esetben, más nyelveíró eszközökkel szemben, hogy a megvalósítás során a megfelelő szemantikát is tudjuk, ugyanabban a nyelvben definiálni. Ezzel ennek az állapotgépnak a futtatása, amelyik a `myfsm` változóban található, egy olyan léptető függvény definiálásával megvalósítható egy lépése, amelyik átveszi a jelenlegi állapotgép állapotot, lépteti egy bemenetnek megfelelően, majd visszaadja az új állapotot – a közben szükséges akciókat végrehajtva. Ennek a lépés függvénynek ismételt hívásával pedig tetszőleges sok lépést lehetséges léptetni a rendszer állapotán.

A léptető függvény megvalósítása erősen függ az állapotgép leíró belső megvalósításától, az általam adott szintén megtalálható a függelékben `fsm_step` néven, míg az üres sorig olvasó ciklikusan léptető függvény a `step_until_empty`. Utóbbi beolvas egy sort, és ha az üres kilép, különben a bemenetet továbbítja a léptető függvénynek, majd rekurzióba kezd. Ezeket ismerve az alábbi kódrészlettel futtathatóvá válik a `myfsm` változóban tárolt állapotgép.

```

1 step_until_empty myfsm

```

Láthatólag ez egy rövid kódrészlet, az alapvetően egyszerű trükkök mind el vannak rejtve a DSL megvalósítása mögé, így az állapotgépet használnak csak azzal kell foglalkoznia, amivel szeretne.

5. fejezet

Jövőbeli fejlesztések és tervek

5.1. Paraméter-kötött függvények

A C4 nyelv által biztosított DSL leírási lehetőség a megfelelő függvények definiálásával történik. Ez az általános megoldás majdnem teljesen szabadon formálhatóvá teszi a nyelvet a DSL írója számára, aki a saját feladatára személyre tudja szabni, hogy a DSL számára megfelelő legyen a szintaktika.

Egy kiemelkedően jól működő struktúra a beágyazott DSL-ek megvalósítására olyan függvények bevezetése, amelyek megfelelő módon hívják egymást, és a szintaktikai elemek megadó külső függvények csak ezeket állítják elő valamilyen closure formájában. Ezeknek a függvényeknek pedig van egy kezdő függvénye, amelyik bevezeti az éppen használt DSL-t. Ez a módszer látható az előző fejezetben adott példák C4 kódjában, alább pedig kiemelve az állapotgép leíró példából a **state** függvény megvalósítása.

```
1 let state/2 { |name exec|
2   { |stm str err fin|
3     let stm2 add_state stm name str err fin
4     &(exec)/4 stm2 0 0 0
5   }
6 }
```

5.1. lista. Az állapotgép DSL **state** függvénye

Ennek a struktúrának előnye, hogy egy-egy DSL használata teljesen egy függvényhívásba limitálódik, hiszen maga a DSL csak ezen a függvényen belül fog megfelelően működni.

Ezt a gondolatot megerősíteni egyik fejlesztési terv olyan függvények bevezetése, amelyek csak más függvények paramétereiként értelmesek. Ezzel megoldódik az esetleges DSL-ek közötti névütközés, hiszen egy-egy kulcsszó csak a megfelelő DSL-en belül kerül feloldásra, így minden DSL a saját verzióját fogja megkapni. Ezeket a függvényeket nevezem paraméter-kötött (parameter-bound) függvényeknek, mert kötve van, hogy melyik másik függvény paraméterében kerülhetnek meghívásra.

```
1 let (state/2 fsm/2) { |name exec|
2   { |stm str err fin|
3     let stm2 add_state stm name str err fin
4     &(exec)/4 stm2 0 0 0
5   }
6 }
```

5.2. lista. Az állapotgép DSL lehetséges paraméter-kötött **state** függvénye

5.2. Scriptelhető statikus analízis eszköz

Az egyik alapvető motiváció a nyelv megvalósítás mögött egy olyan eszköz alapjának létrehozása volt, amelyik lehetővé teszi a felhasználója számára, hogy saját kódrészleteket tudjon futtatni egy C vagy C++ projektben való különböző absztrakt nyelvi elemek keresése és megtalálása esetén.

Példának egy AWK jellegű eszközt lehetséges és érdemes elképzelni, amelyik egy alacsonyabb szinten – a szimpla és egyszerű fájlok szöveges tartalma felett – biztosít futtatható kódrészleteket, amikor egy-egy reguláris kifejezést sikeresen megtalál a bemenetén. Ennek vitathatatlan előnye, hogy általános és minden szöveges fájlra működőképes. Azonban emiatt nem rendelkezik megfelelő mennyiségű szemantikus információval az esetlegesen strukturált bemenetről, így csak a legáltalánosabb, egyszerű szöveg módján tud keresni.

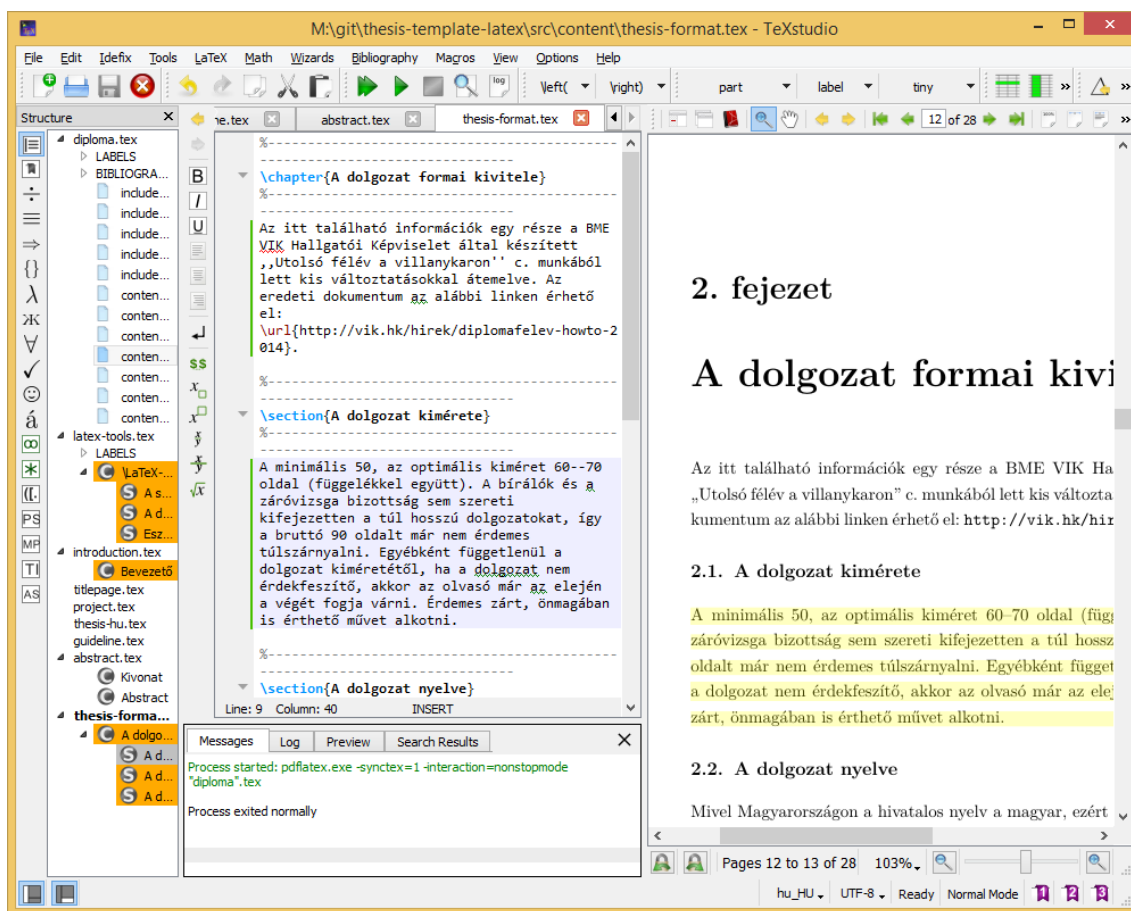
Ez olyan esetekben, ahol a szövegnek szigorúan meghatározott formátuma van kiemelkedően zavaró tud lenni, főleg ha esetlegesen szóköz és egyéb tagoló karakterekre viszont érzéketlen a szöveg formátuma. Ilyen például a programozási nyelvek nagy része (pl. C, C++, Java, stb.) de a kevés nyelv, amelyik tagolásra érzékeny (pl. Python, Haskell) sincs kifejezetten nagy előnyben általánosan. Mivel a nyelvet a konkrét szintaxisával leírva tároljuk a forrásfájlokban, így sok extra zavaró szintaktikai elemet kell tudni megfelelő módon kezelni. Ha szimplán reguláris kifejezésekkel próbálkozunk legjobb esetben nagyságrendileg elbonyolítják a kifejezést, de még az is lehet, hogy teljesen megvalósíthatatlan egy megfelelő kifejezés – például amit szeretnénk az nem egy reguláris nyelvtanra illeszkedő szöveg. A C és C++ nyelvek például szintaktikailag nem különböztetik meg a globális és egy függvényen belüli lokális változókat, sőt C++-ban, még az osztályok tagváltozói is sok esetben azonos formátumúak. Emiatt, ha például azt az egyszerű feladatot szeretnénk megvalósítani, hogy megkeressük, hogy hol és milyen globális változók vannak egy adott projekten belül, akkor nem fogjuk tudni extra szemantikus információ nélkül: tehát az AWK-nál erősebb eszközre van szükség.

Irodalomjegyzék

- [1] Hans J. Boehm: Space efficient conservative garbage collection. *SIGPLAN Not.*, 39. évf. (2004. április) 4. sz., 490–501. p. ISSN 0362-1340.
URL <https://dl.acm.org/doi/10.1145/989393.989442>.
- [2] Hans-J. Boehm – Alan Demers – Mark D. Weiser: A garbage collector for C and C++.
URL <https://www.hboehm.info/gc/>.
- [3] GNU Project: Syntax Rules (Guile Reference Manual). URL https://www.gnu.org/software/guile/manual/html_node/Syntax-Rules.html.
- [4] LLVM Project: The LLVM Compiler Infrastructure Project.
URL <https://llvm.org/>.
- [5] John McCarthy: Recursive functions of symbolic expressions and their computation by machine, Part I. *Commun. ACM*, 3. évf. (1960. április) 4. sz., 184–195. p. ISSN 0001-0782. URL <https://dl.acm.org/doi/10.1145/367177.367199>.
- [6] Philip Hazel: PCRE - Perl Compatible Regular Expressions, 2015. január.
URL <https://www.pcre.org/>.

Függelék

F.1. A TeXstudio felülete



F.1.1. ábra. A TeXstudio $\text{\LaTeX}$ -szerkesztő.

F.2. Válasz az „Élet, a világmindenség, meg minden” kérdésére

A Pitagorasz-tételből levezetve

$$c^2 = a^2 + b^2 = 42. \quad (\text{F.2.1})$$

A Faraday-indukciós törvényből levezetve

$$\text{rot } E = -\frac{dB}{dt} \quad \longrightarrow \quad U_i = \oint_{\mathbf{L}} \mathbf{E} d\mathbf{l} = -\frac{d}{dt} \int_A \mathbf{B} d\mathbf{a} = 42. \quad (\text{F.2.2})$$